

Django REST Framework

Prologue

The Django Web Framework taught you the **essentials to build a web application** and launch it on a browser.

In this course, you will learn the **REST API implementation** using **Django Rest Framework** and cover the concepts of

- Serialization
- Requests and Response
- Class-based view implementation
- Authentication and permission
- Hyperlinking Relations
- Viewsets and Routers
- Schema Documentation

What is REST API?

Before diving into details of Django Rest Framework, let's first understand **REST API**.

An API or **Application Programming Interface** is a code that aids in **communication between two software** applications.

A REST API is an API that uses HTTP requests to **break down** an application into **smaller modules** to perform data operations like

- GET - retrieves a resource
- PUT - modifies a resource
- POST - creates a resource

- DELETE - removes a resource

The Django REST Framework

Django provides a framework to **implement REST API** within web applications through the **Django REST Framework** or DRF.

Django Rest Framework:

- Provides a Web-browsable API for developers.
- Enables authentication policies for secure implementations.
- Provides serialization support for ORM and non-ORM data sources.
- Customizable through Django views.
- Provides extensive documentation and community support.

Rest Framework Supporters

DRF is **used and trusted** by Mozilla, Eventbrite, Red Hat and Heroku.

Prerequisites and Installation

The Django REST framework is built upon the Django Web Framework and available as a Python module.

Hence the prerequisites are the availability of

- Python with version 2.7+
- Django with version 1.10+

DRF can be installed using `pip` command

```
pip install djangorestframework
```

Configuring DRF

To use DRF in a Django Web project, add `rest_framework` to the `INSTALLED_APPS` setting in **settings.py** file

```
INSTALLED_APPS = (  
    ...  
    'rest_framework',  
)
```

DRF enables **authentication to browsable APIs**. To use this feature, add the following to root **urls.py** file.

```
urlpatterns = [  
    ...  
    url(r'^api-auth/', include('rest_framework.urls'))  
]
```

Creating API

Using DRF, you have a variety of options to create effective APIs.

You define the API functionality using views that can be

- **custom functions** where you define each HTTP action
- **Class-based views** with inbuilt HTTP action template.

- **mixins** for reusable API behavior
- **viewsets** for HTTP action with optimized code.

You will get an understanding of each of these methods as you move further along in this course.

With this introduction, let's move on to an interesting topic that defines the **data representation in DRF - Serialization**.

Serializer

When dealing with **complex python data**, such as **querysets and model instances**, serializer helps to

- Convert data to native Python datatypes
- Render data to JSON, XML or other content types.
- Parse data to convert into complex types (deserialization)
- Validate incoming request data.

REST framework serializers are comparable to Django's `Form` and `ModelForm` classes.

- `Serializer` class controls the response output
- `ModelSerializer` class deals with model instances and querysets.

Declaring Serializer

Consider a blog post application class in which you can write blogs.

```
class Blog(object):
    def __init__(self, email, comment):
        self.email = email
```

```
self.comment = comment
self.created = datetime.now()
```

```
blog = Blog(email='fresco@tcs.com', comment='Django is fun')
```

To serialize and deserialize `Blog object data`, you can declare a serializer in `serializers.py` file by subclassing `serializers` within `rest_framework` module.

```
from rest_framework import serializers

class BlogSerializer(serializers.Serializer):
    email = serializers.EmailField()
    comment = serializers.CharField(max_length=160)
    created = serializers.DateTimeField()
```

Serializing Objects

A serializer can be used to **serialize an object** or a **list of objects**.

To translate the model instance `blog` into native Python datatypes, **create a serializer instance** and use the `.data` property

```
>>> serializer = BlogSerializer(blog)
>>> serializer.data
```

Output:

```
{'email': 'fresco@tcs.com', 'comment': 'Django is fun', 'created': '2018-03-03T13:45:02.345832'}
```

Next, **render the data** into any content format. To render as JSON,

```
>>> from rest_framework.renderers import JSONRenderer
>>> json = JSONRenderer().render(serializer.data)
```

```
>>> json
b'{"email":"fresco@tcs.com","comment":"Django is fun","created":"2018-03-03T13:45:02.345832"}'
```

Deserializing Objects

To deserialize `json` object, **parse a stream** into Python native datatypes **using JSONParser**

```
from django.utils.six import BytesIO
from rest_framework.parsers import JSONParser

stream = BytesIO(json)
data = JSONParser().parse(stream)
```

Then, **restore the native datatypes** into a **validated data dictionary**.

```
>>> serializer = BlogSerializer(data=data)
>>> serializer.is_valid()
True
>>> serializer.validated_data
{'comment': 'Django is fun', 'email': 'fresco@tcs.com', 'created': datetime.datetime(2018, 03, 03, 13, 45, 02, 345832)}
```

Saving Object Instance

To save object instance, the serializer's `.create()` and `.update()` methods can be modified.

Consider the previous sample `Blog` data structure. The serializer definition would be

```

class BlogSerializer(serializers.Serializer):
    ... #serializer definition

    def create(self, validated_data):
        return Blog.objects.create(**validated_data)

    def update(self, instance, validated_data):
        instance.email = validated_data.get('email', instance.email)
        instance.comment = validated_data.get('comment', instance.comment)
        instance.created = validated_data.get('created', instance.created)
        instance.save()
        return instance

```

Saving Object Instance

From the definition earlier, you can observe that `serializer.save()` can be used to return object instances.

To create a new serializer instance on `.save()`, pass the data as parameter.

```
>>> serializer = BlogSerializer(data=data)
```

To update an existing object instance `blog` on `.save()`, the object is also passed as a parameter.

```
>>> serializer = BlogSerializer(blog, data=data)
```

Data Validation

To validate serialized data, you need to call `is_valid()` on the serializer instance to check for serialization errors.

If errors occur, the `.errors` property can be used to get a dictionary of the error messages.

A sample is shown below.

```
>>> serializer = BlogSerializer(data={'email': 'fresco', 'comment': 'Django is fun'})
>>> serializer.is_valid()
False
>>> serializer.errors
{'email': [u'Enter a valid e-mail address.']}
```

In the dictionary, the **Key** is the field name and its **value** is a list of error messages.

Nested Representation

To specify **relationships using serializer**, use **Nested object representation**.

The example below shows the relations between a blog and a user, by defining a `UserSerializer` for the user.

```
class UserSerializer(serializers.Serializer):
    email = serializers.EmailField()
    username = serializers.CharField(max_length=100)

class BlogSerializer(serializers.Serializer):
    user = UserSerializer()
    comment = serializers.CharField(max_length=160)
    created = serializers.DateTimeField()
```


If no users are tagged to the blog, you can accept `None` value using

```
user = UserSerializer(required=False)
```

If multiple users are tagged to the blog, the nested representation should be a list.

```
user = UserSerializer(many=True)
```

Serializing a Model



Rather than just a data structure, you frequently work with **models** in Django.

DRF has an **in-built serializer** class to serialize model definitions, the `ModelSerializer`.

The `ModelSerializer` class

- Automatically creates the fields for a model based on the model definition.

- Automatically generates serializer validators.
- `.create()` and `.update()` implementations are included by default.

Next, you will look at how to declare a model-based serializer.

Model Serializer Declaration

A `ModelSerializer` is subclassed from `serializers` and a declaration for the sample `Blog` model looks like,

```
class BlogSerializer(serializers.ModelSerializer):
    class Meta:
        model = Blog
        fields = ('id', 'user', 'comment', 'created')
```

The fields attribute refer to the model's fields.

To serialize all the model's fields, `__all__` can be set on the `fields` attribute

```
fields = '__all__'
```

Model fields can also be excluded from the serializer, use the `exclude` attribute

```
exclude = ('user',)
```

Nested Model Serialization

When using model serializers, nested representations can be shown as key-value pairs using `depth` option.

Consider an example of `Blog` model related to a `User` model. The serializer is

```
class BlogSerializer(serializers.ModelSerializer):
    class Meta:
        model = Blog
        fields = ('id', 'user', 'comment', 'created')
        depth = 1
```

Here, the depth option

- generates nested representation
- set using an integer value
- Indicates the depth of relationships to be traversed

Request Parsing

You have learned the usage of `HttpRequest` in Django - Web Framework to parse a request.

The REST framework extends it using `Request` class for request parsing and authentication.

When an HTTP request is parsed using `Request`,

- The request body can be returned using `request.data`.
- It includes all parsed input content.
- All HTTP methods such as GET, POST, PUT, DELETE, and PATCH are supported.
- It is flexible in request parsing, i.e., whether the input data comes from JSON or a form, it is handled in the same way.

Request Authentication

Each request upon authentication

- Can associate different **authentication policies** for **different API functionalities**.
- Provides **user and token information** for a request.
 - `request.user` - Returns an instance of `django.contrib.auth.models.User`
 - `request.auth` - Provides an instance of the authenticated requests' token.

You will learn more about authentication and permissions in the later topics.

HTML5 Support

HTML5 does not support PUT and DELETE forms.

Response Handling

REST framework provides a `Response` class to render the response content into multiple content types.

- The Response class subclasses Django's `SimpleTemplateResponse`.
- Response objects are initialized with Python native data primitives.
- Rendering of the final response content is determined using content negotiation.
- To enable the browsable API to perform content negotiation, the `APIView` class or `@api_view` decorator is used.

Creating a Response

A response is created using `Response()` class which takes unrendered, python primitive data.

The data **should be serialized** before creating the **response object**.

The following arguments are considered

- `data` - The unrendered Serialized response data.
- `status` - Numeric HTTP response status code.
- `template_name` - To render response into an HTML page with `HTMLRenderer`
- `headers` - A dictionary of HTTP headers to use in the response.
- `content_type` - The content type of the response is set automatically by the renderer.

Accepting Format Suffixed URLs

The API requests do not specify any content type.

However, since REST framework can **handle multiple content** types, you can specify the **nature of content** explicitly in URL Request.

For example, the URL request `http://play.com/api/course/1.json` indicates a JSON input to the API.

This request is handled by adding the `format` keyword in the view definition as shown below:

```
def course_list(request, format=None):
    ...

def course_detail(request, pk, format=None):
    ...
```

Accepting Format Suffixed URLs

Since the View definition accepts multiple content formats, the `urls.py` file can be changed.

Add `format_suffix_patterns` to the URL patterns as shown below to accept format suffixed URLs.

```
from rest_framework.urlpatterns import format_suffix_patterns

urlpatterns = [
    url(r'^course/$', views.course_list),
    url(r'^course/(?P<pk>[0-9]+)$', views.course_detail),
]

urlpatterns = format_suffix_patterns(urlpatterns)
```

Controlling Response Format

The response format can be controlled depending on the request

- Using the Accept header:

```
http http://127.0.0.1:8000/course/ Accept:application/json
http http://127.0.0.1:8000/course/ Accept:text/html
```

JSON format is rendered in the first request, while **text is displayed** in an HTML page in the second request.

- Appending a format suffix:

```
http http://127.0.0.1:8000/course.json
http http://127.0.0.1:8000/course.api
```

JSON format is rendered in the first request, while the response in a browsable API is provided for the second request.

Controlling Request Format

The request format can be controlled using the `Content-Type` header.

POST action using form data

```
$ http --form POST http://127.0.0.1:8000/blog/ comment="post  
action" user="fresco1"  
  
{  
  "id": 2,  
  "user": "fresco1",  
  "comment": "post action",  
}
```

POST action using JSON

```
$ http --json POST http://127.0.0.1:8000/blog/ comment="json  
action" user="fresco2"  
  
{  
  "id": 3,  
  "user": "fresco2",  
  "comment": "json action",  
}
```

Content Negotiation

You heard the term **content negotiation** earlier. Let's understand it's nuances.

Content negotiation refers to the process of **selecting one of the multiple response** representations, based on **client or server preferences**.

It is determined by

- Available renderers
- Priorities of each renderer.
- Client's `Accept:` header.
- Specificity of media types
- The ordering of the configured renderers for a view

Content Negotiation Priorities

Consider an example of an API request, where multiple media types are provided in the `Accept` header.

```
application/json; indent=4, text/html, */*
```

When the request is processed, the response priorities for each of the given media types would be:

1. `application/json; indent=4`
2. `text/html`
3. `/*`

Function Based Views

In the previous topic, you learned about request and response handling, often with a context of rendering response from Views.

You will understand the different ways to implement views on this topic.

If you are writing function based view implementation, the decorator `@api_view` should be used.

This decorator ensures that the view

- receives an instance of the `Request`.
- allows the return of a `Response`
- allows configuring how the request is processed.

Next, you will learn how the `@api_view` decorator can be used.

Using @api_view

A sample view function using `@api_view` is defined below

```
@api_view()
def hello_world(request):
    return Response({"message": "Hello, world!"})
```

The view accepts GET method by default.

To alter the behavior and to use POST requests, the view definition is modified as

```
@api_view(['GET', 'POST'])
def hello_world(request):
    if request.method == 'POST':
        return Response({"message": "Some data", "data": request.data})
    return Response({"message": "Hello, world!"})
```

Class Based Views

REST framework provides built-in class-based views for HTTP method implementation.

The view `APIView` is one such, in which

- `Request` instance is passed to the handler methods.
- `Response` method is returned with **appropriate content negotiation** and correct rendering.
- Any `APIException` is caught automatically and response mediated as appropriate.
- The incoming request **is authenticated with appropriate permission**, prior to handling the request.

Next, let's look at an example to describe the usage of `APIView`.

Usage of `APIView`

When an `APIView` class is used, the request is automatically dispatched to an appropriate handler. Additional API controls can also be set.

Consider a scenario where the admin needs to see all system users. The admin should have token authentication. Users can be listed from `django.contrib.auth.models`

```
from rest_framework.views import APIView
from rest_framework import authentication, permissions
from django.contrib.auth.models import User
...
class ListUsers(APIView):

    authentication_classes = (authentication.TokenAuthenticat
ion,)
    permission_classes = (permissions.IsAdminUser,)
```

```
def get(self, request, format=None):
    usernames = [user.username for user in User.objects.all()]
    return Response(usernames)
```

Generic Views

The generic view class `GenericAPIView` extends `APIView` class to provide standard **list** and **detail view** behavior.

View behavior is controlled by the following attributes.

- `queryset` - Defines the queryset to be used for returning objects.
- `serializer_class` - Serializer class to **validate and deserialize input**, and to **serialize output**.
- `lookup_field` - Performs object lookup of model instances based on model fields
- `lookup_url_kwarg` - The URL keyword argument used for object lookup.

So far, you have learned view-based method handlers. Next, you will learn view based actions.

Using Mixins



The mixin classes imported from `rest_framework.mixins` provides action methods rather than handler methods.

- **ListModelMixin** - Returns a serialized queryset response using `.list` method.
- **CreateModelMixin** - Creates a new model instance using `.create` method and returns a serialized object representation.

- **RetrieveModelMixin** - Returns a serialized object representing an existing model instance using `.retrieve` method.
- **UpdateModelMixin** - Updates an existing model instance using `.update` method and returns a serialized object representation.
- **DestroyModelMixin** - Deletes an existing model instance using `.destroy` method and returns with blank content.

Modifying Mixin Actions

Mixin classes provide **hooks to override default** actions.

Attributes that are **not part of the request** data, but **implicit in the request** can be set.

For example, to create an object based on the requesting user,

```
def perform_create(self, serializer):
    serializer.save(user=self.request.user)
```

Behavioral overrides can also be set before or after saving an object.

The below example sends an email confirmation

```
def perform_update(self, serializer):
    instance = serializer.save()
    send_email_confirm(user=self.request.user, modified=instance)
```

Concrete Views



When you want actions with HTTP method handlers, you combine the `GenericAPIView` and **mixin** to get Concrete Views.

Concrete View classes are imported from `rest_framework.generics`.

The functionalities differ based on the combination.

- **CreateAPIView** - Used for create-only endpoints with a **POST** method handler.

- **ListAPIView** - Represents a **collection of model instances** with **GET** handler for read-only endpoints.
- **RetrieveAPIView** - Represents a **single model instance** with **GET** handler for read-only endpoints.
- **DestroyAPIView** - Used for delete-only endpoints for a single model instance with **DELETE** handler.
- **UpdateAPIView** - Used for update-only endpoints for a single model instance with **PUT** and **PATCH** handlers.

More Concrete Views

Concrete View classes also build upon multiple mixin-based actions.

- **ListCreateAPIView** - Provides GET and POST handlers to represent a collection of model instances with **read-write** endpoints.
- **RetrieveUpdateAPIView** - Provides GET, PUT and PATCH handlers to represent a single model instance with **read or update** endpoints.
- **RetrieveDestroyAPIView** - Provides GET and DELETE handlers to represent a single model instance with **read or delete** endpoints.
- **RetrieveUpdateDestroyAPIView** - Represents a single model instance with **read-write-delete** endpoints and provides GET, PUT, PATCH and DELETE method handlers.

You have learned a lot. It's time to get your hands dirty. Let's move on to hands-on exercises.

Authentication Basics



Authentication is a mechanism that **associates a request with a set of credentials**, such as the **user who posted** the request or the **token signed with** the request.

The credentials are used by **permission**, and **throttling** policies to **permit the request**.

Authentication in a Django view executes prior to the permission and throttling checks, and prior to any other code execution.

Here,

- The `request.user` property is set to an instance of the `contrib.auth.User` class.
- The `request.auth` property is used to identify the request token.

Determining Authentication

The authentication schemes are defined as a list of classes.

Each class is checked from the list, and the first successfully authenticated class is returned. The return value is used to set the `request.user` and `request.auth` parameters.

If no class authenticates,

- **`request.user`** is set to `django.contrib.auth.models.AnonymousUser` instance
- **`request.auth`** is set to **`None`**.

The default authentication schemes can be set using the `DEFAULT_AUTHENTICATION_CLASSES` setting.

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework.authentication.BasicAuthentication',
        'rest_framework.authentication.SessionAuthenticatio
n',
    )
}
```

What happens when a request is denied? Let's have a look.

Unauthorized Response Errors

When a request is **denied permission**, it can be of the two types:

- **Unauthorized** - In this case, a **HTTP 401 Unauthorized** error response is thrown, to **instruct the client** how to authenticate using a `WWW-Authenticate` header.
- **Permission Denied** - In this case, **HTTP 403 Permission Denied** error response is thrown when a request is denied permission, after successful authentication. This **does not include** a `WWW-Authenticate` header.

Next, you will learn the different authentication schemes.

Basic Authentication

This authentication scheme uses **HTTP Basic Authentication**, with a **username** and **password** and provides the following after successful authentication.

- **request.user** set to Django User instance.
- **request.auth** set to None.

When permission is denied for unauthenticated requests, it results in **HTTP 401 Unauthorized** response, with an appropriate **WWW-Authenticate** header.

```
WWW-Authenticate: Basic realm="api"
```

Use BasicAuthentication only over https and ensure that API clients request for username and password at login.

Token Based Authentication

This uses a token-based HTTP Authentication scheme appropriate for client-server setups, like native desktop and mobile clients.

To enable this scheme, add `TokenAuthentication` configuration

```
'DEFAULT_AUTHENTICATION_CLASSES': (
    ...
    'rest_framework.authentication.TokenAuthentication',
)
```

Also add the `rest_framework.authtoken` app to the `INSTALLED_APPS` setting

```
INSTALLED_APPS = (
    ...
    'rest_framework.authtoken'
)
```

Since authtoken is an app provided by DRF, run `python manage.py migrate` to perform database migrations.

Session Based Authentication

This authentication scheme uses Django's default session and is suitable for multiple clients running in the same session context.

In `SessionAuthentication`,

- `request.user` is set to Django User instance.
- `request.auth` is set to None.

`HTTP 403 Forbidden` response is thrown for permission denied requests.

To prevent unsafe HTTP method calls with **SessionAuthentication**, use a **valid CSRF token**.

Use Django's standard login to ensure login pages are protected properly.

Permissions

Permissions use the information from `request.user` and `request.auth` to either grant or deny access to an authenticated request.

Permission checks are executed after authentication checks, prior to any code execution.

Access permissions to a single user can **vary with different sections** of the API.

The most **common permission style** is to **allow access** to an **authenticated** user and **deny access** to any **unauthenticated** user.

You will learn about more permission options later in this topic. Read further to understand how permissions are determined!

Determining Permissions



REST framework permissions are defined as a **list of permission classes**, each checked in order for any exceptions.

When the permissions check fails, the `exceptions.PermissionDenied` or `exceptions.NotAuthenticated` is raised.

HTTP 403 Forbidden response is returned when

- The request is authenticated, but permission denied.

- The request authentication fails, and the high priority authentication class does not use `WWW-Authenticate` headers.

HTTP 401 Unauthorized response is returned when

- The request authentication fails, and the high priority authentication class uses `WWW-Authenticate` headers.

Object Level Permissions

Object level permissions are used to determine access to a user on a particular model instance.

This is run in generic views when `.get_object()` is called and throws `exceptions.PermissionDenied` exception on object access denial.

This permission check can be done explicit using `check_object_permissions`

```
def get_object(self):
    obj = get_object_or_404(self.get_queryset(), pk=self.kwargs["pk"])
    self.check_object_permissions(self.request, obj)
    return obj
```

When using object level permissions, the querysets should be filtered, to ensure users can view only permitted instances.

Setting the Permission Policy

Default permission policy is set globally using `DEFAULT_PERMISSION_CLASSES` setting.

```
REST_FRAMEWORK = {
    'DEFAULT_PERMISSION_CLASSES': (
```

```
        'rest_framework.permissions.IsAuthenticated',
    )
}
```

Authentication policy can be set per-view using the `permission_classes` attribute

```
from rest_framework.permissions import IsAuthenticated
from rest_framework.views import APIView
...
class ExampleView(APIView):
    permission_classes = (IsAuthenticated,)

    def get(self, request, format=None):
        return Response({'status': 'Permission granted'})
```

Permissioning APIs

API permissions are controlled by several **permission** classes.

- **AllowAny** - Provides unrestricted access regardless of authentication.
- **IsAuthenticated** - Denies permission to an unauthenticated user, allows permission to registered users.
- **IsAdminUser** - Used for administrator access, it denies permission to any user unless `user.is_staff` is True.
- **IsAuthenticatedOrReadOnly** - Authenticated users can perform any request, while unauthorized users can only perform read requests.

Model Based Permission

The `DjangoModelPermission` class applies standard model permissions to views that have the `.queryset` property enabled.

Authorization is granted if the user is authenticated and assigned to relevant model permissions.

- To perform **POST** requests, a user requires **add** permission on the model.
- To perform **PUT** and **PATCH** requests, a user requires **change** permission on the model.
- To perform **DELETE** requests, a user requires **delete** permission on the model.

Next, let's catch up on what you have learned on this topic with a hands-on exercise.

Inspecting a Relationship

When you use `ModelSerializer`, the serializer fields and relationships are automatically generated.

To inspect the relationship

- import the serializer class
- instantiate serializer
- print the object representation.

Consider an example of music album with multiple tracks. The relationship between `Album` and `Track` can be seen as below

```
>>> from myapp.serializers import AlbumSerializer
>>> serializer = AlbumSerializer()
>>> print (repr(serializer))
AlbumSerializer():
  album_name = CharField(max_length=100)
  artist = CharField(max_length=100)
```

```
tracks = PrimaryKeyRelatedField(queryset=Track.objects.all())
```

Relationship Representations



The relationship between entities can be represented by

- Using primary keys.
- Connecting entities through hyperlinks.
- Identifying the related entity using a unique slug field.
- Identifying the related entity through default string representation.
- Nesting the related entity within the parent representation.

Let's review some of the most commonly used representations.

Unicode Method Representation

The relationship target is represented by `__unicode__` string, and the serializer uses a `StringRelatedField`.

From the example of album and tracks, the serializer definition can be,

```
class AlbumSerializer(serializers.ModelSerializer):
    tracks = serializers.StringRelatedField(many=True)

    class Meta:
        model = Album
        fields = ('album_name', 'artist', 'tracks')
```

With sample serialized data, it can be represented as

```
{
    'album_name': 'Lost in translation',
    'artist': 'Mordow',
    'tracks': [
        '1: Morning',
        '2: Dusk',
        '3: Rainfall',
```

```
]
}
```

many - argument allows multiple string values.

Primary Key Representation

Here the primary key is the relationship target and is represented using `PrimaryKeyRelatedField`.

Instead of strings, the album and tracks can be related through the primary key.

```
class AlbumSerializer(serializers.ModelSerializer):
    tracks = serializers.PrimaryKeyRelatedField(many=True, read_only=True)

    class Meta:
        ...
```

When serialized, it is represented as,

```
{
  'album_name': 'Lost in translation',
  'artist': 'Mordow',
  'tracks': [
    1,
    2,
    3,
  ]
}
```

- `read_only` - indicates whether the field can be edited or not.

Primary Key Representation

Here the primary key is the relationship target and is represented using `PrimaryKeyRelatedField`.

Instead of strings, the album and tracks can be related through the primary key.

```
class AlbumSerializer(serializers.ModelSerializer):
    tracks = serializers.PrimaryKeyRelatedField(many=True, read_only=True)

    class Meta:
        ...
```

When serialized, it is represented as,

```
{
  'album_name': 'Lost in translation',
  'artist': 'Mordow',
  'tracks': [
    1,
    2,
    3,
  ]
}
```

- `read_only` - indicates whether the field can be edited or not.

Using HyperlinkedModelSerializer

The `HyperlinkedModelSerializer` class uses hyperlinks to represent relationships rather than primary keys.

If a `HyperlinkedModelSerializer` is used

- It includes a URL field by default which serves as the key.
- It does not include `id` field and should be specified explicitly.
- Model Relationships are represented using `HyperlinkedRelatedField`.

An sample definition is shown below.

```
class AlbumSerializer(serializers.HyperlinkedModelSerialize
r):
    class Meta:
        model = Album
        fields = ('url', 'id', 'album_name', 'artist', 'track
s')
```

Fetching Absolute URLs

When an `HyperlinkedModelSerializer` is instantiated, the current request should be included in the serializer context

```
serializer = AlbumSerializer(queryset, context={'request': re
quest})
```

This ensures that the hyperlinks include an appropriate hostname and represents fully qualified URLs.

```
http://www.frescomusic.com/api/tracks/1/
```

rather than relative URLs

```
/tracks/1/
```

Benefits of Absolute URLs

While web frameworks use relative URLs, as a good practice, absolute URLs should be returned for REST APIs.

It offers certain benefits:

- It's more explicit.
- API clients do less work in handling the resource.
- When the string is represented in JSON, its meaning is unambiguous.
- Markup HTML representations with hyperlinks are easy to do.

The utility functions `reverse` and `reverse_lazy` enable returning absolute URLs.

Let's check how they are used.

Using Reverse

You have already encountered URL reversing in Django - Web Framework.

This is the same with REST except that a fully qualified URL is returned using **host** and **port** from the request parameter.

```
from rest_framework.reverse import reverse

class APIRootView(APIView):
    def get(self, request):
        data = {
            ...
            'year-summary-url': reverse('year-summary', args=
[now().year], request=request)
        }
        return Response(data)
```

`request` keyword argument is mandatory.

Using Lazy Reverse

Another method to return a fully qualified URL is to use the `reverse_lazy` module from `rest_framework.reverse`.

This uses the request to determine the host and port. Like `reverse` method, the parameter `request` is added as a keyword argument.

The usage is depicted as

```
api_root = reverse_lazy('api-root', request=request)
```

With this, you should have an understanding of the usage of hyperlinks in API to represent relationships.

In the next topic, you will learn about Routers and Viewsets.

Viewsets

When the functionality of related views is combined in a single class, you get a `ViewSet`.

A `ViewSet`

- Is a class-based View inherited from `APIView`.
- Does not provide `.get()` or `.post()` method handlers, instead provides `.list()` and `.create()` actions.
- Is not wired up to the URL conf, unlike other views.
- Is registered with a `router` class that automatically determines the urlconf.
- minimizes the amount of code

- enforces consistent URL convention across the API.

Creating a Viewset

To create a **viewset** based class, subclass the `ViewSet` class from `rest_framework.viewsets` package.

Consider an example to list all users in the system using a viewset.

- get the user list from the `django.contrib.auth.models.User` module,
- pass the list to a serializer and return the Response.

```
from django.contrib.auth.models import User
from rest_framework import viewsets
...

class UserViewSet(viewsets.ViewSet):
    def list(self, request):
        queryset = User.objects.all()
        serializer = UserSerializer(queryset, many=True)
        return Response(serializer.data)
```

Registering Viewset

A viewset is registered using a **router**. This automatically generates the URL conf.

For the previous **UserViewSet** example, you add the following in `urls.py`

```
from frescoapp.views import UserViewSet
from rest_framework.routers import DefaultRouter

router = DefaultRouter()
```

```
router.register(r'users', UserViewSet, base_name='user')
urlpatterns = router.urls
```

The URL `/users/` is automatically supported by this viewset and need not be explicitly provided.

You will learn routers later in this topic.

ViewSet Actions

ViewSet actions can be list, create, retrieve, update, destroy or partial_update.

The viewset action is checked using `.action` attribute.

Consider the below example

```
def get_permissions(self):
    if self.action == 'list':
        permission_classes = [IsAuthenticated]
    else:
        permission_classes = [IsAdmin]
    return [permission() for permission in permission_classes]
```

Here, the user can only perform list action while the the admin can perform all actions

Custom Routing Actions

If you have custom methods that you need to be routed to, use the `@detail_route` or `@list_route` decorators.

- `@detail_route` use when methods return a single instance such as setting a user password.

- `@list_route` used for methods operating on a list of objects, such as listing users by last login time.

The example below shows the usage of `@list_route` for listing users by last login

```
from rest_framework.decorators import list_route
...
@list_route()
def recent_users(self, request):
    recent_users = User.objects.all().order('-last_login')
    serializer = self.get_serializer(recent_users, many=True)
    return Response(serializer.data)
```

Reversing Action URLs

When using custom actions, the URL can be obtained using the `.reverse_action()` method. This,

- Automatically passes the view's request object
- Prepends the `url_name` with the `.basename` attribute.

`basename` is provided by the router during ViewSet registration.

If you have defined a custom function `set_password` with a router using `@detail_route`, the URL can be obtained as shown below.

```
>>> view.reverse_action('set-password', args=['1'])
'http://localhost:8000/api/users/1/set_password'
```

Next, you will review some Viewset APIs.

ViewSet APIs

ViewSet has standard API's for different actions.

- `ViewSet` - inherits from `APIView` with **standard attributes** such as `permission_classes`, `authentication_classes`. It **does not** provide any **action implementation** by default.
- `GenericViewSet` - inherits from `GenericAPIView` with `get_object`, `get_queryset` methods, but **does not include** any actions by default.
- `ModelViewSet` - inherits from `GenericAPIView` and **includes** action implementations such as `list`, `retrieve`, `create`, `update` and `destroy`.
- `ReadOnlyModelViewSet` - inherits from `GenericAPIView` and **includes** implementations for 'read-only' actions, `list` and `retrieve`.

Routers

Routers help to **tag the view logic** to a set of URLs, thus enabling **automatic URL routing** to Django.

Consider an example of router which **registers two viewsets**, one for **Users** and one for **User accounts**. This is written in `urls.py` as

```
from rest_framework import routers

router = routers.SimpleRouter()
router.register(r'users', UserViewSet)
router.register(r'accounts', AccountViewSet)
urlpatterns = router.urls
```

`register` method takes two mandatory parameters

- prefix signifying the URL prefix to use.
- viewset class

More on Routers

An optional third argument can also be specified.

- `base_name` - indicates the URL names that are created.

The following URL patterns are generated from the previous example.

```
URL pattern: ^users/$ Name: 'user-list'
URL pattern: ^users/{pk}/$ Name: 'user-detail'
URL pattern: ^accounts/$ Name: 'account-list'
URL pattern: ^accounts/{pk}/$ Name: 'account-detail'
```

'user-list', 'user-detail', 'account-list' and 'account-detail' are the basenames.

Using include with Routers

The `router` instance has a `.url` attribute which helps to provide the list of URL patterns like 'user-list', 'user-detail'.

If your code has other view implementations, the router urls can be appended using Django's `include()` function.

```
urlpatterns = [
    url(r'^user-reset/$', UserResetFormView.as_view()),
    url(r'^$', include(router.urls)),
]
```

Router Types

You have seen the router implementations earlier. Routers are of two types

`SimpleRouter`

- Includes routes for list, create, retrieve, update, partial_update and destroy actions.

- The viewset uses `@detail_route` or `@list_route` for additional methods to be routed
- Appends a trailing slash by default
- Matches values containing any characters other than slashes and period characters.

`DefaultRouter` has all features of a `SimpleRouter` and in addition

- Includes a default API Root view containing hyperlinks to all list views

With this knowledge, let's move on to a hands-on exercise.

Schema

API Schema is a tool to generate reference documentation that describes

- resources that are available for API
- URLs and their representation
- what operations are supported

The Python package `coreapi` is used to render the schema information and generate API documentation.

You will learn more about `coreapi` usage later in this topic.

Schema Output Formats

When Core API is used, the schema representation format is **Core JSON**, which is rendered using the `renderers.CoreJSONRenderer` module.

Alternate schema formats such as Open API, JSON HyperSchema or API Blueprint can be generated using specific codecs.

For example, you can use `openapi_codec` package to encode or decode to the **Open API (Swagger)** format

```

from rest_framework import renderers
from openapi_codec import OpenAPICodec

class SwaggerRenderer(renderers.BaseRenderer):
    media_type = 'application/openapi+json'
    format = 'swagger'

    def render(self, data, media_type=None, renderer_context=None):
        codec = OpenAPICodec()
        return codec.dump(data)

```

Creating a Schema

Schema can be auto-generated or can be manually specified

To manually specify a schema you create a Core API Document as shown.

```

schema = coreapi.Document(
    title='Course Search API',
    content={ ... }
)

```

By processing a list of URL patterns and compiling the Core API document, the `SchemaGenerator` class automatically creates schema as shown.

```

generator = schemas.SchemaGenerator(title='Course Search AP
I')
schema = generator.get_schema()

```

Adding a Schema View

To include a schema in your project, use the `get_schema_view()` function in the `urls.py` file.

```
from rest_framework.schemas import get_schema_view

schema_view = get_schema_view(title="Course Search API")

urlpatterns = [
    url('^$', schema_view),
    ...
]
```

title argument is provided in the above example.

Next, you will look at other possible arguments.

Schema Arguments

Let's understand the `get_schema_view()` arguments through an example.

```
schema_view = get_schema_view(
    title='Course Search API',
    url='https://www.play.org/api/',
    urlconf='myproject.urls'
    renderer_classes=[CoreJSONRenderer, APIBlueprintRenderer]
    patterns=schema_url_patterns,
)
```

- **title** - a descriptive title for the schema definition.
- **url** - to pass a canonical URL for the schema.
- **urlconf** - represents the URL path to generate API schema.

- `renderer_classes` - to render the API root endpoint
- **patterns** - to restrict schema to specific URL patterns

Core API

Using Core API,

- Schema is represented as a `Document` containing information about the API.
- Represents available API interactions using `Link` objects.
- Each link includes a URL, HTTP method, and list of Fields instances

Core API is available as a command line client using `coreapi-cli` package installed through `pip`

```
$ pip install coreapi-cli
```

Core API Actions

Core API client can be used to load a schema.

```
$ coreapi get http://127.0.0.1:8000/schema/

<Blog API "http://127.0.0.1:8000/schema/">
  blogs: {
    list()
    read(id)
  }
  users: {
    list()
    read(id)
  }
```

Without authentication, the schema representation has read only endpoints

List Action

To list the API contents, use `list` argument

For example, to list all blogs, use

```
$ coreapi action blogs list

[
  {
    "url": "http://127.0.0.1:8000/blogs/1/",
    "id": 1,
    "comment": "post action",
    "user": "fresco1",
    "created": "2018-03-03T13:45:02.345832"
  },
  ...
]
```

Adding Authentication

To get all the possible API interactions using coreapi, you need to

Authenticate coreapi

```
$ coreapi credentials add 127.0.0.1 <username>:<password> --a
uth basic
Added credentials
127.0.0.1 "Basic <...>"
```

Reload schema

```
$ coreapi reload
<Blog API "http://127.0.0.1:8000/schema/">
  blogs: {
    create(comment, user)
    delete(id)
    list()
    update(id, comment, user)
    read(id)
  }
  ...
```

Next, try experimenting yourself with coreapi.

Epilogue

Congratulations on learning the course with high spirit.

You managed to sail through the Django REST Framework by learning

- How serialization helped to render data in REST.
- How REST-based web requests and responses are handled.
- The usage of in-built views and custom views.
- The concept of viewsets and routers and to leverage them for standard REST APIs.
- Secure ways of deploying REST API.
- Documenting the API through Schema.

Your ride was rough, but you proved to be tough.